

```
#include <SoftwareSerial.h>
#include <Servo.h>

const int button = 2;
const int buzzer = 8;
const int servoPin = 9;

const int bluetoothRX = 10;
const int bluetoothTX = 11;

SoftwareSerial bluetooth(bluetoothRX, bluetoothTX);
Servo pencil;

const int UP = 90;
const int DOWN = 35;

const int DOT_TIME = 200;
const int DASH_TIME = 600;

String morse = "";

String code[] = {
  ".-", "...", "-.-.", "-..", ".", "-.-.", "-.-.",
  "...", ":", "-.-.-", "-.-.", "-.-.", "-.-.", "-.-.",
  "-.-.", "-.-.", "-.-.-", "-.-.", "...", "-.-", "-.-.",
  "...", "-.-", "-.-.", "-.-.-", "-.-."
};

unsigned long pressStart;
unsigned long lastRelease;

bool buttonWasPressed = false;

// ----- SETUP -----

void setup() {
  pinMode(button, INPUT_PULLUP);
  pinMode(buzzer, OUTPUT);

  Serial.begin(9600);
  bluetooth.begin(9600);

  pencil.attach(servoPin);
  pencil.write(UP);

  bluetooth.println("Ready");
  bluetooth.println("Button: Morse to English");
  bluetooth.println("Bluetooth: English to Morse");
  bluetooth.println();
}

// ----- MAIN LOOP -----

void loop() {
  readButton();
  translateMorse();
  readBluetooth();
}

// ----- MORSE TO ENGLISH -----

void readButton() {
  bool pressed = digitalRead(button) == LOW;

  // Button has just been pressed
  if (pressed && !buttonWasPressed) {
    pressStart = millis();
    buttonWasPressed = true;
  }

  // Button has just been released
  if (!pressed && buttonWasPressed) {
    int pressTime = millis() - pressStart;

    if (pressTime < 300) {
      morse += ".";
      beep(DOT_TIME);
    } else {
      morse += "-";
      beep(DASH_TIME);
    }

    lastRelease = millis();
    buttonWasPressed = false;
  }
}

void translateMorse() {
  if (morse != "" && !buttonWasPressed) {
    int pauseTime = millis() - lastRelease;

    // Pause of 1 to 2.5 seconds: end of letter
    if (pauseTime > 1000 && pauseTime < 2500) {
      bluetooth.print(morse);
      bluetooth.print(" = ");
      bluetooth.println(findEnglish(morse));

      morse = "";
    }

    // Pause of 2.5 seconds or more: end of word
    if (pauseTime >= 2500) {
      bluetooth.print(morse);
      bluetooth.print(" = ");
      bluetooth.println(findEnglish(morse));

      bluetooth.println("WORD SPACE");

      morse = "";
    }
  }
}

char findEnglish(String pattern) {
  for (int i = 0; i < 26; i++) {
    if (code[i] == pattern) {
      return 'A' + i;
    }
  }

  return '?';
}

// ----- ENGLISH TO MORSE -----

void readBluetooth() {
  while (bluetooth.available()) {
    char letter = bluetooth.read();

    // Ignore Enter key characters
    if (letter == '\n' || letter == '\r') {
      continue;
    }

    // Convert lowercase to uppercase
    if (letter >= 'a' && letter <= 'z') {
      letter = letter - 32;
    }

    // Space between words
    if (letter == ' ') {
      bluetooth.println("SPACE");
      delay(2100);
      continue;
    }

    // Check whether the character is A to Z
    if (letter >= 'A' && letter <= 'Z') {
      String pattern = code[letter - 'A'];

      // English first, Morse second
      bluetooth.print(letter);
      bluetooth.print(" = ");
      bluetooth.println(pattern);

      makeMorse(pattern);
    }
  }
}

// ----- SERVO AND BUZZER -----

void makeMorse(String pattern) {
  for (int i = 0; i < pattern.length(); i++) {
    if (pattern[i] == '.') {
      makeDot();
    }

    if (pattern[i] == '-') {
      makeDash();
    }
  }

  // Gap after one letter
  delay(400);
}

void makeDot() {
  pencil.write(DOWN);

  tone(buzzer, 1000);
  delay(DOT_TIME);
  noTone(buzzer);

  pencil.write(UP);
  delay(DOT_TIME);
}

void makeDash() {
  pencil.write(DOWN);

  tone(buzzer, 1000);
  delay(DASH_TIME);
  noTone(buzzer);

  pencil.write(UP);
  delay(DOT_TIME);
}

void beep(int length) {
  tone(buzzer, 1000);
  delay(length);
  noTone(buzzer);
}
```